4 Colour Drawing for 570/991es+

Step 1:
Stage 1 data loader.
Use basic overflow in LINEIO comp mode
Use this hackstring:

Bước 1:
Trình tải dữ liệu giai đoạn 1.

Sử dụng tràn cơ bản trong chế độ biên dịch LINEIO
Sử dụng chuỗi hackstring này:

X = Σ(X , 1 , ×$_{10}$ 9 9
[CALC] [=]
[LEFT] [DEL] [DEL] [DEL] [1] [CALC] [=] [LEFT]

91x{[1]} (press [1] 91 times) (nhấn [1] 91 lần)

(-) cs30 0 0 Ran# F 1 0 (-) cs30 0 0 cv12 = 0 0 sin⁻(, 0 0 <UNTYPEABLE2f> <UNTYPEABLE84> Abs(cs17 1 0 cv22 Pol(1 0 cv12 = 0 0 sin(2 0 0 <UNTYPEABLE18> ʳ ° 4 1 0 <UNTYPEABLE7e> A 1 0 ∫(cs23 0 0 tan⁻(D 0 0 cs01 cs01 , ° 1 0 cv02 **e**^(0 0 <UNTYPEABLE18> ʳ cs10 D 0 0 3 3 3 3 cv24 M 1 0 <UNTYPEABLEb8> ʳ M ʳ (-) cs23 0 0 ∫(÷ 0 0 . ᵍ log(÷ 0 0 9 0

For “UNTYPEABLE” chars, you have to use timer byte, so spam characters from that position until the M indicator lights up, and then press DEL 11 times, and then press the given character below and wait the duration in multiples of 0.5 seconds (every time the cursor toggles on/off is 0.5s) once the amount of time has been waited, you must immediately press LEFT DEL to “lock it in”, and then repeatedly press DEL to bring the char into position from where you spammed.

Đối với các ký tự “KHÔNG THỂ NHẤN”, bạn phải sử dụng byte hẹn giờ, vì vậy hãy nhấn liên tục các ký tự từ vị trí đó cho đến khi đèn báo M sáng lên, sau đó nhấn phím DEL 11 lần, rồi nhấn ký tự được chỉ định bên dưới và chờ khoảng thời gian là bội số của 0,5 giây (mỗi lần con trỏ bật/tắt là 0,5 giây). Sau khi chờ đủ thời gian, bạn phải ngay lập tức nhấn phím DEL TRÁI để “khóa” ký tự, và sau đó nhấn phím DEL liên tục để đưa ký tự vào vị trí mà bạn đã nhấn liên tục.

UNTYPEABLE2F = .+1
UNTYPEABLE84 = pi+2
UNTYPEABLE18 = cs15 + 9
UNTYPEABLE7E = cs29 + 4
UNTYPEABLEB8 = cs33 + 1

[AC] [LEFT] [=]

Step 2:
Magic.

Bước 2:
Ảo thuật.

EXE DEL
) +
3 DEL
2 DEL
- ln(
8 ÷
+ ^2
. ^2
6 $\times_{10}$
$\times_{10}$ Frac
2 DEL
2 DEL
+ 8
. EXE
3 DEL
2 DEL
EXE DEL
) +
3 DEL
2 DEL
÷ ln(
3 ÷
))
5 ÷
× ×
4 DEL
2 DEL
2 DEL
× DEL
- ^-1
+ 8
. EXE
3 DEL
2 DEL
, $\times_{10}$

4 DEL
2 DEL
2 DEL
(×
5 ÷
AC CALC
^2 Int
2 DEL
2 DEL
+ 8
. EXE
3 DEL
2 DEL
, $\times_{10}$
4 DEL
2 DEL
2 DEL
, (-)
6 ÷
^2 Int
$\times_{10}$ Frac
2 DEL
2 DEL
+ ^2
+ ^2
EXE EXE
1 DEL
2 DEL
2 DEL
2 DEL
2 DEL
, $\times_{10}$
4 DEL
2 DEL
2 DEL
2 DEL
2 DEL
7 Ans
, DEL
2 DEL
2 DEL
2 log(
× DEL
2 DEL

2 DEL
8 DEL
(+
3 DEL
2 DEL
M+ DEL
8 DEL
2 DEL
2 DEL
6 $\times_{10}$
× ENG
2 DEL
2 DEL
8 DEL
(+
3 DEL
2 DEL
^2 Int
$\times_{10}$ Frac
2 DEL
2 DEL
1 $\times_{10}$
5 ÷
M+ DEL
3 9
3 DEL
2 DEL
6 $\times_{10}$
6 DEL
3 DEL
2 DEL
, $\times_{10}$
4 DEL
2 DEL
2 DEL
- Ans
5 ÷
÷ ln(
4 DEL
3 DEL
2 DEL
5 ln(
sin(sqrt(
2 DEL

2 DEL
EXE EXE
1 ÷
^2 Int
$\times_{10}$ Frac
2 DEL
2 DEL
EXE EXE
1 ÷
1 9
$\times_{10}$ logxx
2 DEL
2 DEL
2 DEL
2 DEL
7 9
. DEL
2 DEL
2 DEL
hyp Int
5 ÷
^2 Int
$\times_{10}$ Frac
2 DEL
2 DEL
+ ^2
$\times_{10}$ Ans
7 9
. DEL
2 DEL
2 DEL
hyp Int
5 ÷
^2 Int
$\times_{10}$ Frac
2 DEL
2 DEL
+ ^2
- ^-1
7 9
. DEL
2 DEL
2 DEL
hyp Int

5 ÷
^2 Int
$\times_{10}$ Frac
2 DEL
2 DEL
- ^-1
+ ^2
7 9
. DEL
2 DEL
2 DEL
^2 Int
$\times_{10}$ Frac
2 DEL
2 DEL
$\times_{10}$ Ans
$\times_{10}$ Ans
- Int
$\times_{10}$ Frac
6x{2 DEL}
, $\times_{10}$
4 DEL
2 DEL
2 DEL
+ Int
+ Int
hyp 8
× ENG
2 DEL
2 DEL
1 9
$\times_{10}$ logxx
2 DEL
2 DEL
+ 3
6 ÷
0 8
$\times_{10}$ logxx
2 DEL
2 DEL
- Ans
6 DEL
3 DEL
2 DEL

- Ans
1 DEL
2 DEL
2 DEL
^2 Int
$\times_{10}$ Frac
2 DEL
2 DEL
EXE EXE
1 ÷
- Ans
6 DEL
3 DEL
2 DEL
^2 Int
$\times_{10}$ Frac
2 DEL
2 DEL
+ ^2
0 EXE
M+ DEL
3 9
3 DEL
2 DEL
1 9
$\times_{10}$ logxx
2 DEL
2 DEL
. DEL
6 ÷
0 8
$\times_{10}$ logxx
2 DEL
2 DEL
- Ans
6 DEL
3 DEL
2 DEL
(sin(
1 DEL
2 DEL
2 DEL
, $\times_{10}$
4 DEL

2 DEL
2 DEL
3 Int
+ ^2
hyp 8
× ENG
2 DEL
2 DEL
. DEL
3 DEL
2 DEL
2 DEL
, $\times_{10}$
4 DEL
2 DEL
2 DEL
hyp 7
+ ^2
hyp 8
× ENG
2 DEL
2 DEL
, $\times_{10}$
4 DEL
2 DEL
2 DEL
(-) EXE
Ans DEL
6 DEL
3 DEL
14x{2 DEL}
+ 8
. EXE
3 DEL
2 DEL
EXE DEL
) +
3 DEL
2 DEL
0 ×
6 ÷
- ^-1
+ ^2
÷ ln(

4 DEL
3 DEL
2 DEL
÷ ln(
× S<>D
2 DEL
2 DEL
× DEL
- ^-1
÷ ln(
3 ÷
))
5 ÷
2 DEL
2 DEL
AC EXE
3 9
(AC
+ ^2
2 DEL
2 DEL
1 9
$\times_{10}$ logxx
2 DEL
2 DEL
(-) EXE
5 ÷
0 8
$\times_{10}$ logxx
2 DEL
2 DEL
5 ÷
3 AC
× S<>D
5 EXE
9 ÷
3 DEL
$\times_{10}$ Frac
hyp 2
0 ^2
+ 2
. ^2
2 9
- ^-1

4 ÷
+ ^2
+ (-)
124x{+ ^2}
(+
- Ans
7 ÷
) +
7 ÷
+ 5
7 EXE
2 DEL
2 DEL
1 9
$\times_{10}$ logxx
2 DEL
2 DEL
0 DEL
5 ÷

How to use

Up/down/left/right = move cursor
1/2/3 = light grey/dark grey/black

Notes:
The cursor starts at 0,0 meaning it is on the indicator line, to be at the top left of the screen, press down once.
There is no way to turn off a pixel / make it less darker than it already is, nor is there a way to clear the screen, feel free to add functionality to it if you want, the code is down below.

Cách sử dụng

Lên/xuống/trái/phải = di chuyển con trỏ
1/2/3 = xám nhạt/xám đậm/đen

Lưu ý:
Con trỏ bắt đầu ở vị trí 0,0, nghĩa là nó đang ở trên đường chỉ báo. Để đến góc trên bên trái màn hình, hãy nhấn giữ một lần.

Không có cách nào để tắt một pixel / làm cho nó nhạt hơn độ tối hiện tại, cũng như không có cách nào để xóa toàn bộ màn hình. Bạn có thể thêm chức năng nếu muốn, mã nguồn ở bên dưới.

CASIO
NATURAL-V.P.A.M.
ESP TERMINAL
>FX-ES(MS)
TOTALYNCE
SHIFT
ALPHA
MODE SETUP
ON
REPLAY
SOLVE
CALC
DEC
HEX
BIN
OCT
log
ln
FACT
Abs
hyp
sin
cos
tan
STO
RCL
ENG
S⇔D
M+
CONST
CONV
CLR
INS
OFF
DEL
AC
MATRIX
VECTOR
nPr
nCr
STAT
CMPLX
BASE
Pol
Rec
Rnd
Ran#
RanInt
DRG▸
Ans

How to run on emu without 1000+ keypress

For casioemumsvc:
Goto rop->injection hex->slot 1
Set injection address to 8430
Copy paste

Cách chạy trên trình giả lập mà không cần nhấn hơn 1000 phím

Đối với casioemumsvc:
Vào rop->injection hex->slot 1
Đặt địa chỉ injection thành 8430
Sao chép và dán

ee 54 31 30 d0 87 00 03 aa 44 30 30 63 1e 31 30 ee 54 31 30 f2 82 52 84 9c 32 30 30 4c 01 63 1e 31 30 a0 32 30 30 76 84 e2 b2 30 30 63 1e 31 30 a0 32 30 30 8c 85 b2 44 30 30 00 00 e0 2f 30 30 30 30 a0 32 30 30 30 30 c2 2a 30 30 9a 4c 30 30 36 53 31 30 ec 36 30 30 aa 46 30 30 36 53 31 30 b2 44 30 30 a5 84 ec 6c 31 30 aa 34 31 30 a0 32 30 30 b8 84 f2 32 31 30 d8 73 30 30 e0 80 b2 44 30 30 e0 80 6a 4f 30 30 30 30 70 2c 30 30 e6 84 b2 44 30 30 00 ff 70 2c 30 30 e6 84 b2 44 30 30 00 01 70 2c 30 30 e6 84 b2 44 30 30 01 00 70 2c 30 30 b2 44 30 30 ff ff 0a 44 30 30 30 30 30 30 a0 32 30 30 08 08 41 46 30 30 6a 4f 30 30 5e 85 68 4f 30 30 b8 34 31 30 b8 2f 30 30 b2 44 30 30 e0 80 b8 34 31 30 b2 44 30 30 00 20 ec 6c 31 30 6a 4f 30 30 2c 85 68 4f 30 30 b8 34 31 30 c8 2f 30 30 a0 32 30 30 10 00 41 46 30 30 2c 31 30 30 a0 32 30 30 40 00 41 46 30 30 a0 32 30 30 50 89 34 31 30 30 30 30 30 30 30 30 30 30 30 30 30 30 63 1e 31 30 ee 54 31 30 7e 85 01 00 f2 32 31 30 f2 48 30 30 4c 01 f2 82 52 84 30 30 d6 6c 0e 00 30 30 6a 4f 30 30 50 84 68 4f 30 30 84 19 48 25 88 31 44 3b 05 5d 03 6b 01 83 00 8b

Goto watch
Click pause

Set pc to 0x04B8E set lr to 0x04B8E set SP to 0x8430

Go back to rop, press inject

Go back to watch, click resume

Paint program is now running

Vào chế độ xem
Nhấp vào tạm dừng
Đặt pc thành 0x04B8E, đặt lr thành 0x04B8E, đặt SP thành 0x8430

Quay lại rop, nhấn inject

Quay lại chế độ xem, nhấp vào tiếp tục

Chương trình vẽ hiện đang chạy

Source code:

Mã nguồn:

```
home:
      xr0 = 0x87d0, 0x0300
      memzero
      set lr
      xr0 = adr_of [-352] main_run, adr_of main_run
      memcpy, pop er0
      0x014C
      set lr
main_run:

      er0 = adr_of kkode
      getscancode_nodelay
      set lr
      er0 = adr_of key_switch
      er2 = 0x0000
      ea = er0, [ea] |= r3, pop er4
      0x3030
      pop er0
kkode:
```

```
      0x3030
      r1 |= r0, [8100] = r1
      r0 = r1
      r1 = 0
      ea_switchcase
      r0 = [ea] # r0 now holds the index into the jumptable
      r1 = 0
      er2 = adr_of [-27] up
      er0 += er2
      er2 = er0, er0 += er4 # idc about er0 now
      er0 = adr_of jadr
      [er0] = er2
      er8 = 0x80E0 # address for up down right left to right to
      er2 = 0x80E0 # address of coord for setting pixel to write to
      pop er14
jadr:
      0x3030 # ting ting
      jpop er14

      adr_of [-2] cnt
up:
      er2 = 0xFF00
      jpop er14

      adr_of [-2] cnt
down:
      er2 = 0x0100
      jpop er14

      adr_of [-2] cnt
right:
      er2 = 0x0001
      jpop er14
left:
      er2 = 0xFFFF
cnt:
      [er8] += er2,pop xr8
      0x30303030
      er0 = 0x0808
      delay
      goto restore
p3:
      er0 = [er2], r2 = 9
```

```
      set_pixel # same as p1, nothing special
      er2 = 0x80E0
p2:
      er0 = [er2], r2 = 9
      er2 = 0x2000
      er0 += er2 # make y position go out of bounds into our own second
screen buffer
      goto spix
p1:
      er0 = [er2], r2 = 9
spix:
      set_pixel_unbound
rend:
      er0 = 0x0010 # adjust this, pwm stuff
      delay
      render
      er0 = 0x0040 # adjust this, pwm stuff
      delay
      er0 = 0x8950
      render er0
      0x3030303030303030
      0x30303030

restore:
      set lr
      xr0 = adr_of length, 0x01, 0x00
      [er0] = er2
      qr0 = 0x014C, adr_of [-352] main_run, adr_of main_run, 0x30, 0x30
      0x6CD6
length:
      0x000E
      0x3030
      goto main_run
key_switch:
      0x84 # up
      0x19

      0x48 # down
      0x25

      0x88 # right
      0x31
```

```
0x44 # left
0x3B

0x05 # 3
0x5D

0x03 # 2
0x6B

0x01 # 1
0x83

0x00 # default
0x8B
```

Cre: Totalynce